

# PAPER ANALYSIS: [RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning](#)

Most LLM code generation starts with a simple contract:

*"Give me the problem, and I will give you the code."*

That works surprisingly well. But it breaks down in an important situation: **what happens when the code is wrong?**

A conventional LLM can see the error message. It can even be prompted to "try again." But seeing feedback and knowing what to do with that feedback are fundamentally different capabilities.

The paper [RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning](#) is built around this distinction. Its central idea is simple:

**Core Thesis:** Don't just give the LLM feedback at inference time. Train the LLM with feedback so that it learns to use the environment's response to change its next action.

## 1. The Problem: Generating Another Answer Is Not the Same as Repairing an Answer

Imagine asking an LLM:

*"Write a program that solves this graph problem."*

It generates:

**Solution 1**

You run it:

TIMEOUT

Now you ask:

*"Try again."*

The model generates:

## Solution 2

The problem is that **Solution 2 might barely differ from Solution 1**. It may have seen the word TIMEOUT, but that doesn't mean it has learned:

*"My current algorithm is too slow, therefore I need a fundamentally different algorithm."*

This is the gap the paper focuses on. An LLM already knows how to generate code. What it does not reliably know is:

How Should my next generation depend on what happened to my previous generation?

The authors point out that previous work found something counterintuitive: **under a fixed inference budget, independent sampling can actually beat iterative repair**. In other words, you might be better off asking the LLM for three unrelated answers than asking it to repeatedly repair one answer.

That observation creates the paper's central research question:

*Can we train an LLM so that iterative generation becomes genuinely useful?*

## 2. Section 1 — Introduction: The Key Idea Is

### "Grounding"

The authors start from a broader view of LLM agents. An agent isn't necessarily a one-shot mapping:

Problem → Answer

Instead, it operates inside an environmental feedback loop:

Problem



LLM



Action



Environment



Feedback



LLM



Another action



...

The environment becomes part of the problem. For code, this environment is particularly convenient because it is **executable**.

1. The LLM generates a program.
2. The computer runs it.
3. The computer returns a concrete execution status:
  - PASS
  - FAIL
  - TIMEOUT
  - EXCEPTION
  - OUT OF MEMORY

The authors call this **execution feedback**. That feedback contains information that wasn't necessarily available in the original prompt.

A prompt might say:

*"Solve this programming problem."*

Only after execution do you discover:

*"Your implementation times out on this particular test input."*

The environment has revealed something about reality that the model couldn't know with certainty from the prompt alone. That is what the paper means by **grounding in feedback**: the model's next behavior should be tied directly to what actually happened in the environment.

### 3. Section 2.1 — Turn Code Generation into an Interaction

This is the first major engineering decision. Instead of a single static call:

Problem → LLM → Code

The authors set up an interactive rollout:

Problem

↓

LLM

↓

Code 1

↓

Public tests

↓

Feedback

↓

LLM

↓

Code 2

↓

Public tests

↓

Feedback

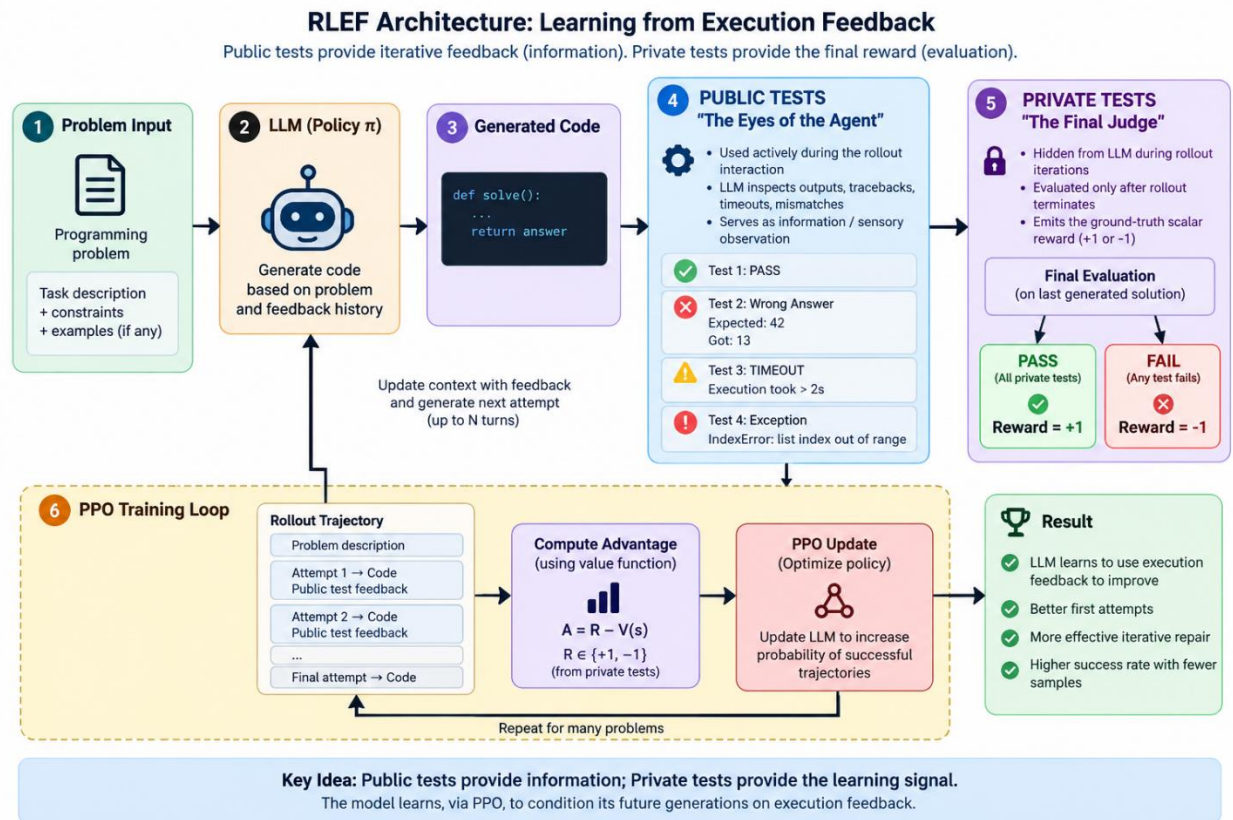
↓

...

The original problem, previous solutions, and their feedback remain in the conversational context so the model can reason about its history. This creates an RL-like sequential environment.

## 4. Public Tests vs. Private Tests

This separation is one of the most elegant parts of the setup.



### Public tests are the eyes of the agent

They are used during the interaction:

Code → Public test → Feedback → next generation

The model sees their detailed results:

- Mismatched input/output: Input X failed | Expected: Y | Observed: Z
- Performance errors: Execution took too long
- Runtime exceptions: IndexError / stack trace

The prompt templates explicitly accommodate wrong-answer, exception, timeout, and out-of-memory diagnostics.

### Private tests are the judge

The model **never** sees private tests during the iterative process. Once the rollout terminates, the final candidate is executed against the private test suite:

- **All Private Tests Pass:**  $\text{Reward} = +1$
- **Any Private Test Fails:**  $\text{Reward} = -1$

## Why separate them?

1. **Prevents Reward Hacking:** If private tests were visible, the model could trivially overfit or hardcode edge cases instead of fixing the underlying algorithm.
2. **Computational Efficiency:** Rollout feedback stays fast and cheap by executing a compact public suite during iteration.

Public tests = **Information**  $\iff$  Private tests = **Evaluation**

## 5. A Complete RLEF Episode Walkthrough

Here is the lifecycle of a single training episode:

[Attempt 1]

Problem: "Find a solution that runs within a strict time limit."

↓

LLM generates Code A

↓

Public execution:

Test 1 → PASS

Test 2 → PASS

Test 3 → TIMEOUT

↓

Feedback generated: "Your code failed test 3. Execution took too long."

[Attempt 2]

LLM receives Feedback + History

↓

LLM generates Code B (algorithm optimized)

↓

Public execution:

Test 1 → PASS

Test 2 → PASS

Test 3 → PASS

↓

Model stops early (all public tests passed)

[Final Evaluation]

Code B evaluated against Private Tests → PASS

↓

Final Reward = +1

## The Trajectory vs. The Reward

Trajectory = [Problem → Code A → TIMEOUT → Code B → Pass ]

Reward = +1

The paper does **not** feed PPO a textual explanation such as "*Code B is good because you responded correctly to a timeout.*" Instead, PPO receives the raw trajectory and its scalar outcome, learning statistically across thousands of episodes. The authors formulate the optimization objective as a scalar task reward combined with a KL regularization penalty.

## 6. Section 2.2 — Turn the Interaction into an RL

### Problem

The authors formalize this interactive loop as a Markov Decision Process (MDP):

| RL Concept       | RLEF Implementation                           |
|------------------|-----------------------------------------------|
| Policy ( $\pi$ ) | The LLM generating responses                  |
| Action ( $a$ )   | Complete code response generated by the model |
| Environment      | The sandboxed code execution                  |

|                                       |                                                                    |
|---------------------------------------|--------------------------------------------------------------------|
|                                       | environment                                                        |
| <b>Observation (<math>s_t</math>)</b> | Problem description + conversation history<br>+ execution feedback |
| <b>Episode</b>                        | One complete multi-turn solve attempt                              |
| <b>Reward (<math>R</math>)</b>        | Final correctness score (scalar: $+1$ or $-1$ )                    |

At every step  $t$ , the LLM conditions on the full history to decide:

$$\pi(a_t | s_t) : \text{"Given what I know now, what should I generate next?"}$$

## 7. The Most Important Subtlety: Feedback Is Not Reward

A common point of confusion is treating error messages as negative rewards. They are not:

- **Feedback is an observation ( $s_t$ ):** It tells the agent *what happened* in the environment so it can adapt.
- **Reward is the objective signal ( $R$ ):** It tells the learning algorithm *whether the overall trajectory succeeded*.

Code 1 → TIMEOUT (Observation, not penalty)

↓

Model adjusts strategy

↓

Code 2 → Evaluated on Private Tests

↓

PASS → Reward +1 | FAIL → Reward -1

## 8. Where Does the Value Function Fit?

In PPO actor-critic architectures:

- The **Policy** ( $\pi$ ) asks: "What tokens / code should I generate?"
- The **Value Function** ( $V$ ) asks: "How promising is this current state?"

Suppose the agent reaches a state:

$$\text{State } s = \text{Problem} + \text{Failed Code 1} + \text{TIMEOUT Feedback}$$

The value function estimates the expected future return:

$$V(s) \approx 0.4$$

- 

If the trajectory continues and ultimately passes ( $R = +1$ ), the advantage is **positive** ( $A > 0$ ), reinforcing the actions taken.

- If it fails ( $R = -1$ ), the advantage is **negative** ( $A < 0$ ), discouraging that branch.

The learned value function serves directly as the baseline for generalized advantage estimation in PPO.

## 9. Hybrid Granularity: Tokens vs. Whole Programs

LLMs generate code token by token, but code execution is atomic—the environment can only evaluate complete programs.

ENVIRONMENT LEVEL (Turn-level)

One complete program generated  $\longrightarrow$  ACTION EXECUTED

MODEL LEVEL (Token-level)

token<sub>1</sub>  $\longrightarrow$  token<sub>2</sub>  $\longrightarrow$  ...  $\longrightarrow$  token<sub>500</sub>

The authors adopt a hybrid design:

- **Policy ( $\pi$ )**: Updated at the **token level**.
- **Value Function ( $V$ )**: Evaluated at the **turn level** (whole response).

The value of the response is estimated as a single unit, assigning one unified advantage to the tokens that produced that response.

### Ablation Evidence (Llama 3.1 8B, 1@3)

| Formulation                | Test Pass Rate |
|----------------------------|----------------|
| Turn-Level Value Function  | 16.0%          |
| Token-Level Value Function | 13.7%          |

Evaluating the response as a cohesive unit aligns value estimation with environmental execution dynamics.

## 10. Why PPO Instead of Supervised Fine-Tuning (SFT)?

A natural question is: why not simply collect successful repair trajectories and train via ordinary next-token prediction?

The authors tested this explicitly. They collected **313,639 successful trajectories** and trained an SFT baseline primarily on the successful final turns.

### PPO vs. SFT Results

| Model Size    | SFT (Test) | RLEF (Test) | Relative Gain |
|---------------|------------|-------------|---------------|
| Llama 3.1 8B  | 10.0%      | 16.0%       | +60.0%        |
| Llama 3.1 70B | 27.2%      | 40.1%       | +47.4%        |

Simply imitating successful solutions fails to teach the model how to recover when it goes off-distribution. The exploration, counterfactual feedback, and policy gradient optimization loop are crucial.

## 11. Section 3 — Empirical Results

The headline benchmarks on CodeContests demonstrate substantial improvements:

| Model                   | Valid Pass Rate | Test Pass Rate |
|-------------------------|-----------------|----------------|
| Llama 3.1 8B (Initial)  | 8.9%            | 10.5%          |
| Llama 3.1 8B (RLEF)     | 17.2%           | 16.0%          |
| Llama 3.1 70B (Initial) | 25.9%           | 27.5%          |
| Llama 3.1 70B (RLEF)    | 37.5            | 40.1%          |

## Sample Efficiency

Traditional code generation approaches often rely on high sample counts (e.g., pass@100 or pass@1000) through repeated independent sampling. RLEF makes each individual sample significantly more effective, matching or exceeding prior baselines with substantially smaller inference budgets.

## 12. The Decisive Experiment: Destroying the Feedback

How do we prove the model actually reads and understands the feedback, rather than simply learning a generic heuristic like *"on turn 2, generate something completely different"*?

The authors tested this by injecting **random execution feedback from unrelated problems**:

Actual Event: Your solution timed out on large array.

Injected Feedback: `IndexError: string index out of range on line 12.`

## Result

- Performance collapsed, and error recovery rates plummeted.
- If RLEF had merely learned to diversify on subsequent turns, random feedback would have had little effect.
- Because incorrect feedback severely degraded performance, the model demonstrated that it **actively conditions its edits on the semantic content of the execution trace**.

## 13. Behavioral Inspection: What Happens After Errors?

Analyzing diffs between sequential attempts reveals clear behavioral differences:

### Standard Instruction-Tuned Model

Attempt 1 (Code A) —► TIMEOUT —► Attempt 2 (Code A')

Result: Microscopic edits, variable renaming, or identical repetitions.

## RLEF-Trained Model

Attempt 1 (Code A) —► TIMEOUT —► Attempt 2 (Code B)

Result: Algorithmic overhaul (e.g., switching  $O(N^2)$  nested loop to  $O(N \log N)$  with binary search).

RLEF-trained models:

1. Make fewer mistakes on the initial attempt.
2. Fix errors significantly more reliably.
3. Make larger, more structural code edits when needed.

## 14. Independent Sampling vs. Iterative Repair

Returning to the paradox noted in the introduction:

*Can iterative repair actually beat independent sampling under an equal sample budget?*

- **Before RLEF:** Independent 1-turn sampling often beat multi-turn iterative prompting on the 8B model.
- **After RLEF:** Iterative repair strategies with 3, 5, and 10 turns consistently outperformed independent sampling.

### The Sweet Spot

- **5 turns** yielded the highest performance.
- Extending to **10 turns** provided diminishing returns under equal sample budgets.

RLEF does not imply "*infinite turns are always better*"; it makes bounded turns meaningfully productive.

## 15. Is the Improvement Really From Feedback, or Simply From Better Code Generation?

The paper is careful about this distinction. RLEF improves the first response too.

So part of the gain comes from:

- **Better first attempt**

Another part comes from:

- **More diverse subsequent attempts**

But the random-feedback experiment shows that another part really is:

- **Targeted feedback-based repair**

The authors explicitly acknowledge this mixture in the conclusion. So it would be incorrect to summarize the result as:

*"RLEF only teaches self-repair."*

It doesn't. It improves the **whole trajectory**.

## 16. Section 4 — Related Work: What Was New Here?

The authors are careful about claiming novelty. Before RLEF, there were already systems doing:

Generate code → Execute → Repair

There were also sophisticated agentic frameworks that added:

- Planning
- Test generation
- Execution
- Repair
- Multiple specialized LLM calls

The problem was that such systems could require dozens of LLM calls per solution, making them computationally expensive.

There was also earlier RL work on code synthesis using execution rewards. So RLEF's contribution isn't:

- *"Use RL for code."*
- *"Use execution to repair code."*

The decisive contribution is the combination:

Execution Environment

+

Textual Execution Feedback

+  
Iterative Generation  
+  
Online RL (PPO)  
+  
One Unified Model

The authors emphasize that execution feedback is **not merely a scalar reward**. It is also provided to the model as text, allowing the model to learn code synthesis and code repair together.

That is the conceptual niche of RLEF.

## 17. Section 5 — Conclusion: What Did They Actually Prove?

The authors' final claim is fairly focused. RLEF gives an LLM the ability to:

**Ground future generations in environment feedback.**

In their code setting, that means:

Act → Observe what actually happened → Adapt → Act again

They demonstrate substantial improvements on CodeContests, better sample efficiency, generalization to HumanEval+ and MBPP+, and meaningful use of execution feedback. But the paper also makes clear what it doesn't solve.

## 18. Major Bottleneck #1: You Need an Evaluatable Environment

This is perhaps the biggest limitation. For code:

Code → Compiler / Interpreter → Tests → Objective Feedback

This pipeline is straightforward to construct. But consider:

*"Write a good business strategy."*

- What is the environment?
- How do you automatically emit a scalar reward like  $+1$  or  $-1$ ?
- And what textual diagnostic feedback should you provide?

The authors explicitly note that iterative execution feedback naturally requires test cases, which may not always be available. They point toward automatic test generation as a future direction. This is a major generalization bottleneck.

## 19. Major Bottleneck #2: Long-Horizon Tasks

The paper mainly studies:

One Problem  $\longrightarrow$  One Solution  $\longrightarrow$  Repair That Solution

It does not solve a much larger agentic problem such as:

Large Task



Decompose Task



Plan



Implement Component A



Test



Implement Component B



Test



Integrate



Observe



Repair

↓

...

The authors explicitly identify task decomposition and larger environments as future work. That is a much harder RL problem because the horizon becomes longer and credit assignment gets significantly harder.

## 20. Major Bottleneck #3: Feedback Quality Matters

The random-feedback experiment makes this very clear:

- **Good feedback** → **Good repair**
- **Wrong feedback** → **Bad repair**

So a feedback-grounded agent is only as useful as its environment's feedback. This suggests a broader principle:

**An intelligent agent plus a poor evaluator can still behave poorly.**

In code, the execution environment is unusually clean. In real-world agents, feedback may be:

- Delayed
- Ambiguous
- Incomplete
- Noisy
- Expensive
- Or unavailable

The paper doesn't solve that general problem; it deliberately chooses code because execution feedback is unusually well-defined.

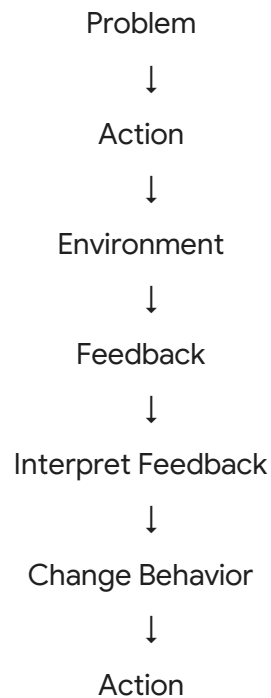
## 21. The "Brain" of the Paper

After everything discussed, RLEF reduces to this:

### Standard LLM Paradigm

Problem  $\longrightarrow$  Good Answer

### RLEF Interactive Loop



And the crucial learning signal is:

**Did this entire interaction eventually succeed?**

*(not: "Was every intermediate step correct?")*

That is why RL is useful here. The model doesn't need an engineer to manually hardcode:

*"When you see TIMEOUT, switch from algorithm A to algorithm B."*

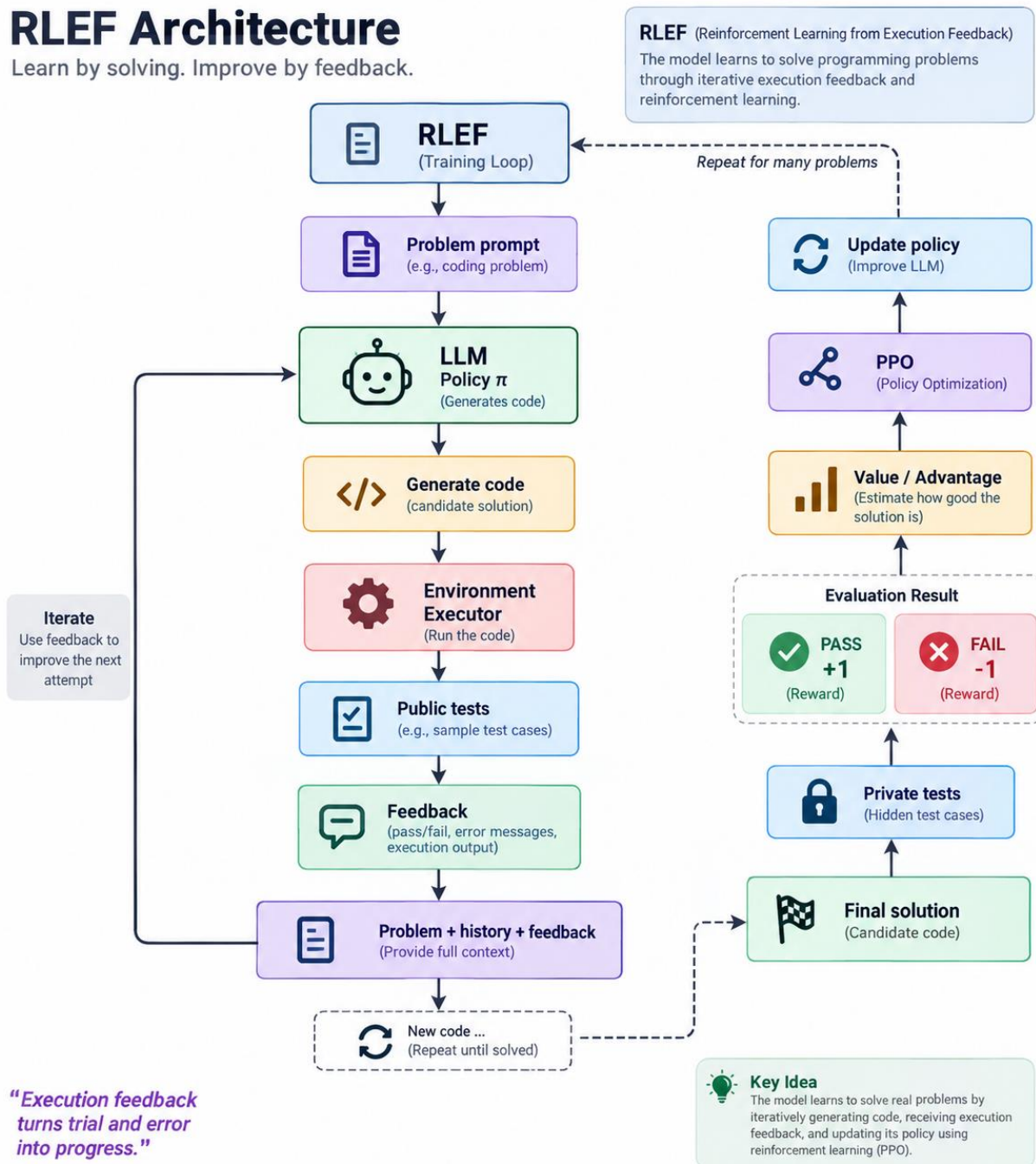
Instead, through many trajectories, it learns a statistical relationship between:

Context + Feedback + Chosen Response  $\implies$  Eventual Success

## 22. One Diagram to Remember

### RLEF Architecture

Learn by solving. Improve by feedback.



## Final Takeaway

The most useful way to remember this paper is not:

*"RLEF is a PPO technique for code generation."*

That's technically true, but it misses the point. The real idea is:

**LLMs are good at producing actions, but autonomous agents also need to learn from what happens after those actions. RLEF uses code execution as an environment and PPO as the learning mechanism to teach the LLM to condition its future actions on execution feedback.**

That transforms the LLM from:

*"Give me another answer."*

Into something closer to:

*"My previous action caused this outcome. Given that information, what should I do differently now?"*

And that is the core insight of the paper.

The paper's most convincing evidence is not even the headline benchmark number. It is the combination of three results:

1. **RLEF > Ordinary Iterative Prompting**
2. **RLEF > SFT on Successful Trajectories**
3. **Correct Feedback  $\gg$  Random Feedback**

Together, these support the central claim that training the model to interact with the environment can teach genuine feedback-grounded behavior, rather than merely making it produce more candidate programs.

**One final caveat:** the study demonstrates this capability in a particularly friendly environment—code, where tests provide automatic feedback. The harder research question is whether the same principle can work when feedback is noisy, delayed, subjective, or has to be generated by another model or by the environment itself. The authors explicitly leave that broader extension open.